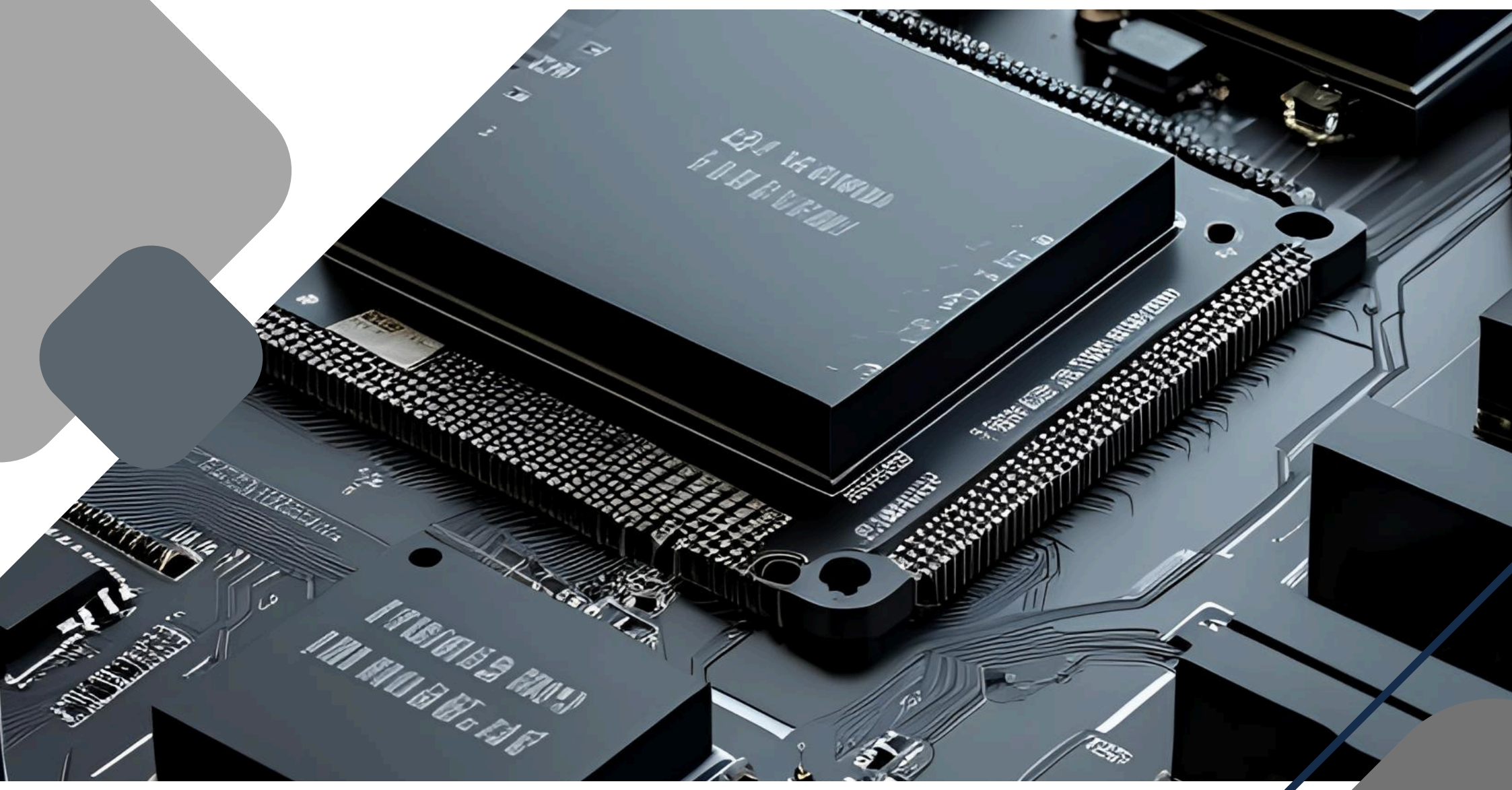


TWO-LAYERED

SMART SECURITY

An IOT Based Security System

Prepared by:

Ayan Shaukat
Undergraduate Electronics Engineer

Noor Fatima
Undergraduate Electronics Engineer

TABLE OF CONTENTS

SECTION.1

REQUIREMENT SPECIFICATIONS

1.1. PROJECT OVERVIEW

Objective: To detect unauthorized intrusion using dual-sensor logic and notify users in real-time using Blynk IoT platform, while logging the incident time for future reference.

1.2. USER PERSPECTIVE

1.2.1. User Interface Description

Users interact with the system through the **Blynk IoT Mobile App** or **Web Dashboard**. Key features include:

- Real-time notifications.
- Intrusion logs with timestamps.
- Status indication using virtual LEDs.

Usage Flow

1. Device powered on; LED remains green.
2. If motion is detected:
    - Blue light alert + notification for presence.
3. If intrusion is suspected:
    - Red LED + buzzer + alert + timestamp log.
4. User can reset the system using a physical button or app.

1.3. DESIGNER PERSPECTIVE

1.3.1. System Architecture

Hardware Layer:

- **Controller**: ESP32 Dev Kit
- **Sensors**: PIR Motion Sensor (HC-SR501), Vibration Sensor (SW-420)
- **Actuators**: RGB LED, Active Buzzer, Pushbutton Switch
- **Platform**: Blynk IoT Cloud (WiFi-based)

Software Layer:

- Programming Language: Arduino C++
- Platform: Blynk IoT (Mobile App + Web Console)
- Libraries: WiFi.h, BlynkSimpleEsp32.h, TimeLib.h

1.4. FUNCTIONAL REQUIREMENTS

ID	REQUIREMENT DESCRIPTION
FR1	System shall detect motion using a PIR sensor.
FR2	System shall detect vibration using the SW-420 sensor.
FR3	System shall send Blynk notifications for specific events.
FR4	System shall control RGB LED colors based on detection logic.
FR5	System shall trigger an active buzzer on dual-sensor detection.
FR6	System shall allow alarm deactivation using a physical switch.
FR7	System shall log and send intrusion timestamps to Blynk for monitoring.

1.5. CONSTRAINTS

CATEGORY	CONSTRAINT DESCRIPTION
Hardware Cost	Should remain within $5–10 per unit
Power	ESP32 operates at 3.3V; components must be compatible
Connectivity	Requires stable 2.4GHz WiFi for Blynk communication
Timing	Buzzer duration: 15 seconds (unless interrupted)
Response Time	LED/buzzer and notification response within 1 second of detection

1.6. NON-FUNCTIONAL REQUIREMENTS

- **Reliability**: False positives minimized by layered detection.
- **Latency**: Notification and logging delay less than 2 seconds.
- **Security**: Only authenticated Blynk users can access device data.
- **Maintainability**: Code is modular for ease of updates and feature expansion.

1.7. FUTURE ENHANCEMENTS

- Integration with **camera module** to capture image on intrusion.
- Can use the reed switch to confirm the suspicion of break-in
- **Firebase or Google Sheets** support for more detailed logs.
- Voice alerts using DFPlayer Mini or TTS module.
- Battery backup for power outages.

SECTION.2

SYSTEM DESGINS

2.1. INTRODUCTION

This document outlines the design architecture of the “Two-Layered Smart Security System,” a smart intrusion detection and alerting solution. It uses the ESP32 Dev Kit microcontroller to interface with sensors and transmits alerts to the Blynk IoT platform. The design choices focus on reliability, cost-efficiency, and rapid prototyping.

2.2. SYSTEM DESIGN OPTIONS

2.2.1. ESP8266 NodeMCU

Although widely used, ESP8266 offers fewer GPIO pins and lacks native support for multi-sensor setups requiring simultaneous tasks (e.g., buzzer + LED + Blynk). Its memory and GPIO limitations restrict its use for more complex logic and peripheral interfacing.

2.2.2. Raspberry Pi Pico W

It supports WiFi and offers good GPIO availability but lacks the community IoT integration support compared to ESP32 and is relatively new in the ecosystem.

2.2.3. ESP32 Dev Kit

This option uses the ESP32 microcontroller, which supports multiple digital IOs, built-in WiFi/Bluetooth, dual-core processing, and great community/library support. It allows efficient multitasking such as triggering buzzers, controlling LEDs, and sending HTTP requests to Blynk servers simultaneously.

2.3. OPTION EVALUATION

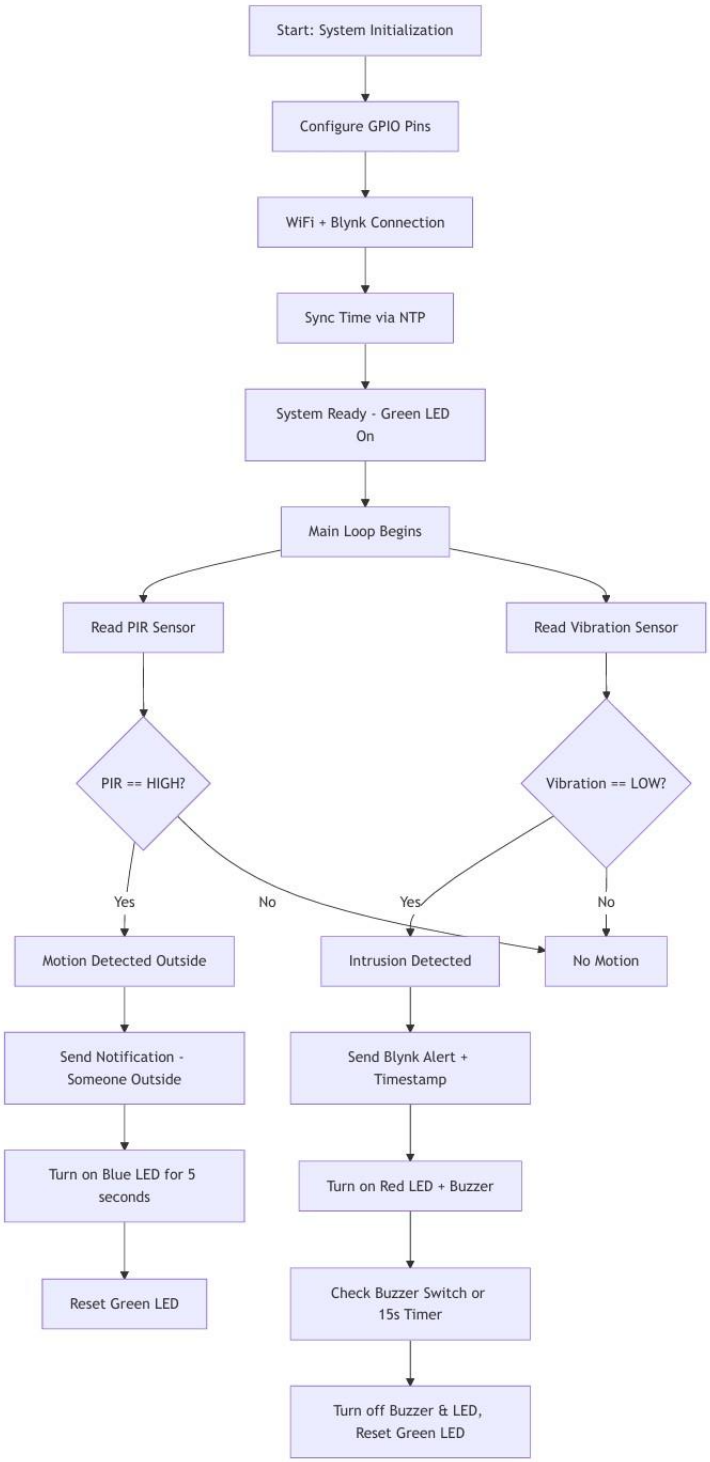
OPTION	COST	EASE OF USE	LIBRARY SUPPORT	WIFI INTEGRATION	SUITABILITY
ESP8266 NodeMCU	Low	High	Excellent	Seamless	Moderately Suitable
Raspberry Pi Pico W	Moderate	Medium	Developing	Built-in	Moderately Suitable
ESP32 Dev Kit	Low	High	Excellent	Built-in & Robust	Highly Suitable

2.4. FINAL DESIGN CHOICE

The **ESP32 Dev Kit** was selected for the project due to its native WiFi capabilities, GPIO availability, dual-core processing, and vast open-source support. It handles concurrent events like sensor inputs, LED signals, and cloud updates efficiently.

2.5. METHODOLOGY

2.5.1. Block Diagram



2.5.2. System Flow Description

1. **Idle Mode**: RGB LED stays **green**.
2. **PIR Sensor Triggered Alone**:
    - LED turns **blue** for 5 seconds.
    - Notification sent: *"Someone is present outside your place!"*
3. **Both PIR and Vibration Triggered**:
    - LED turns **red**.
    - Buzzer turns on for 15 seconds (can be turned off using switch).
    - Notification: *"Alert!! Someone is trying to break in."*
    - Intrusion time is logged to Blynk.
4. **Vibration Sensor**: No action is taken.

SECTION.2

TEST PROCEDURE

3.1. INTRODUCTION

This test procedure describes the validation and functionality of a home intrusion detection system developed using an ESP32 microcontroller. The system incorporates a PIR sensor to detect motion, a SW-420 vibration sensor to identify physical tampering, a buzzer for audible alerts, colored LEDs to indicate system status, and a push button for manual control. The device connects to the internet via Wi-Fi and integrates with the Blynk IoT platform to provide real-time notifications and timestamped intrusion logs.

3.2. TESTING OF COMPONENTS

3.2.1. PIR Sensor

The PIR sensor is tested by simulating motion in front of the sensor, such as moving a hand or walking past it. When only the PIR sensor is triggered, the system logs a general motion detection message to the serial monitor and Blynk. It also turns on a blue LED for five seconds as a visual indicator. This confirms that the sensor is correctly detecting motion events and that the corresponding response logic is functioning as expected.

3.2.2. Vibration Sensor (SW-420)

The vibration sensor uses active-low logic, meaning it outputs a LOW signal when vibration is detected. It should be triggered by tapping or shaking the surface it's attached to. The system only activates the full alarm sequence if both the vibration sensor and the PIR sensor are triggered simultaneously. This results in a red LED being turned on and the buzzer sounding an alert. A notification with a timestamp is sent through Blynk, and the intrusion event is displayed in the terminal widget. This verifies that the vibration sensor is functioning correctly.

3.2.3. ESP32 Wi-Fi Module

The ESP32's Wi-Fi capability is initialized at start-up. The serial monitor displays the connection status and prints the local IP address once connected. The configTime() function is used to sync time from NTP servers, which enables accurate timestamps to be generated during intrusion events. Successful testing of this component involves confirming that the device connects to Wi-Fi, retrieves a valid time, and can use that time in intrusion logs and Blynk updates.

3.2.4. Blynk Web and Mobile Dashboard

The Blynk platform is integrated to provide mobile alerts and a visual display of events. Testing involves connecting the ESP32 to the Blynk cloud using the authentication token and verifying communication with the mobile app. Upon motion or intrusion detection, the system sends push notifications and displays timestamped messages in the Blynk terminal. The terminal output should begin with a formatted heading and show each intrusion event with its corresponding date and time. This confirms that Blynk is working properly for both event notifications and UI updates.

3.3 INTEGRATION TESTING

3.3.1. ESP32 with Sensors and Indicators

This stage validates the integration of the PIR sensor, vibration sensor, LEDs, buzzer, and push button with the ESP32. When only the PIR sensor is triggered, the system should activate the blue LED for a short duration and log the motion to Blynk. When both the PIR and vibration sensors are triggered, the system should activate the red LED and buzzer, log a critical intrusion alert to Blynk, and display a timestamp. The green LED serves as the system's default "ready" indicator and should turn off during alerts and return to HIGH once the system resets. This confirms that all hardware components are interacting correctly based on the defined logic.

3.3.2. ESP32 and Blynk Communication

In this phase, the ESP32's communication with the Blynk cloud is tested in a live network environment. Upon detecting events, the system should send formatted event logs to the terminal and trigger push notifications. The push button should be able to cancel the buzzer during an intrusion. Observing this sequence confirms that the ESP32 is able to send and receive real-time data through the Blynk platform, and that the user interface responds correctly to events.

3.3.3. Full Stack Hardware and Cloud Integration

A complete system test is conducted by simulating both motion and vibration. When both sensors are triggered, the system should execute the full alarm sequence, send real-time alerts to the Blynk app, display the intrusion log in the terminal, and reset appropriately. The buzzer must turn off either after 15 seconds or upon button press. Repeating this sequence multiple times should produce consistent behavior and no failed transmissions. This ensures the stability of the entire stack including sensors, indicators, Wi-Fi, and cloud communication.

3.3.4. Live Dashboard Verification

During this phase, the Blynk mobile app is observed to verify live responsiveness. When events are triggered, the app should display push notifications without noticeable delay and update the terminal log with accurate timestamps. The label widget in the Blynk interface should reflect the intrusion status or duration, and the behavior of the LEDs and buzzer should match what is displayed in the app. This step confirms that the mobile interface accurately reflects real-world activity detected by the ESP32.